

Rotinas armazenadas

Duas procedures de escrita e uma function de consulta em 06_etapa2_procedures.sql

Procedure e function no PostgreSQL

Os três nomes seguem o prefixo pedido no enunciado, mas os objetos não são do mesmo tipo. As rotinas que alteram dados são chamadas com **CALL**. O cálculo de espera é uma **FUNCTION ... RETURNS TABLE**, pois precisa devolver linhas dentro de um **SELECT**.

Objeto	Chamada	Resultado
sp_registrar_atendimento_completo	CALL	id do atendimento pelo parâmetro INOUT
sp_calcular_tempo_medio_espera	SELECT	uma linha por unidade
sp_reajustar_escala	CALL	quantidade de escalas movidas pelo INOUT

sp_registrar_atendimento_completo

A procedure recebe os dados do atendimento e um array JSONB. Cada item representa um procedimento realizado. **id_procedimento** é obrigatório; quantidade assume 1, faturado assume falso e data_hora_inicio assume o horário do atendimento quando não foi informada.

```
CALL sp_registrar_atendimento_completo(  
    '2026-08-03 09:00', 40, 1, 11, 6, 2,  
    '[{"id_procedimento": 5, "tempo_real_minutos": 28}]'::jsonb,  
    NULL  
);
```

A lista precisa ser um array e não pode estar vazia. A rotina valida paciente, residente, preceptor, unidade e cada procedimento antes de concluir. Um item sem identificador é recusado. Repetir o mesmo procedimento viola a chave primária composta de Procedimento_Realizado; repetições devem ser registradas em quantidade.

Como a criação circular funciona. O atendimento entra primeiro e os procedimentos entram na sequência, dentro da mesma transação. Os constraint triggers são diferidos e verificam a cardinalidade no COMMIT. O estado intermediário é permitido; o estado confirmado precisa ter ao menos um procedimento.

Não há COMMIT no corpo. O CALL participa da transação aberta pelo chamador. Se o terceiro item falhar, a exceção desfaz o atendimento e os itens anteriores. Pela API, o caminho suportado para criação é **POST /atendimentos/completo**.

sp_calcular_tempo_medio_espera

A função encontra o menor `data_hora_inicio` de cada atendimento e subtrai a `data_hora` de chegada. Depois agrupa por unidade e arredonda a média para uma casa decimal. atendimentos sem unidade, sem início preenchido ou com horário anterior à chegada não entram na conta.

Unidade no seed	Atendimentos considerados	Espera média
Enfermaria A	4	33,8 min
Ambulatorio	2	19,0 min
Pronto-Socorro	4	10,3 min
UTI Adulto	5	5,8 min

As colunas retornadas são `unidade_id`, `nome_unidade`, `atendimentos_considerados` e `espera_media_minutos`. Os nomes evitam ambiguidade entre parâmetros de saída do PL/pgSQL e colunas do esquema. A API expõe o resultado em **GET /atendimentos/tempo-medio-espera**.

sp_reajustar_escala

A procedure move a escala de um residente entre dois pares de dia e turno. Antes de consultar a origem, ela bloqueia a linha do residente com **SELECT ... FOR UPDATE**. Essa linha existe mesmo quando não há escala, por isso funciona como ponto estável de serialização.

```
SELECT 1 FROM Residente
WHERE id_profissional = p_id_residente
FOR UPDATE;
```

Chamadas concorrentes para a mesma pessoa passam uma de cada vez. Depois que a primeira confirma, a segunda relê a origem: se a escala já foi movida, devolve zero. Isso elimina o caso em que as duas chamadas anunciavam a mesma movimentação.

- Valida a existência do residente, os nomes de dia e turno e a diferença entre origem e destino.
- Recusa destino já ocupado e mais de uma linha na origem, situação incompatível com a nova UNIQUE.
- Incrementa `escala.versao` no mesmo UPDATE que altera dia e turno.
- Devolve a quantidade alterada; zero significa que não havia plantão na origem.

A barreira final é **uq_escala_residente_dia_turno**. A chamada HTTP correspondente é **POST /escalas/reajustar**.

Tratamento de erro e testes

O router executa CALL ou SELECT na mesma sessão SQLAlchemy usada pela aplicação. Em erro de banco, faz rollback e converte o SQLSTATE em resposta HTTP. Conflitos de unicidade viram 409; referências, parâmetros e checks inválidos viram 400.

tests/test_db.py contém 21 casos diretos para as três rotinas. O teste adversarial do reajuste abre duas conexões e exige os resultados 0 e 1: uma chamada move a escala e a outra, após o lock, constata que não há mais linha na origem.